AOS Warranty Analytics on AWS

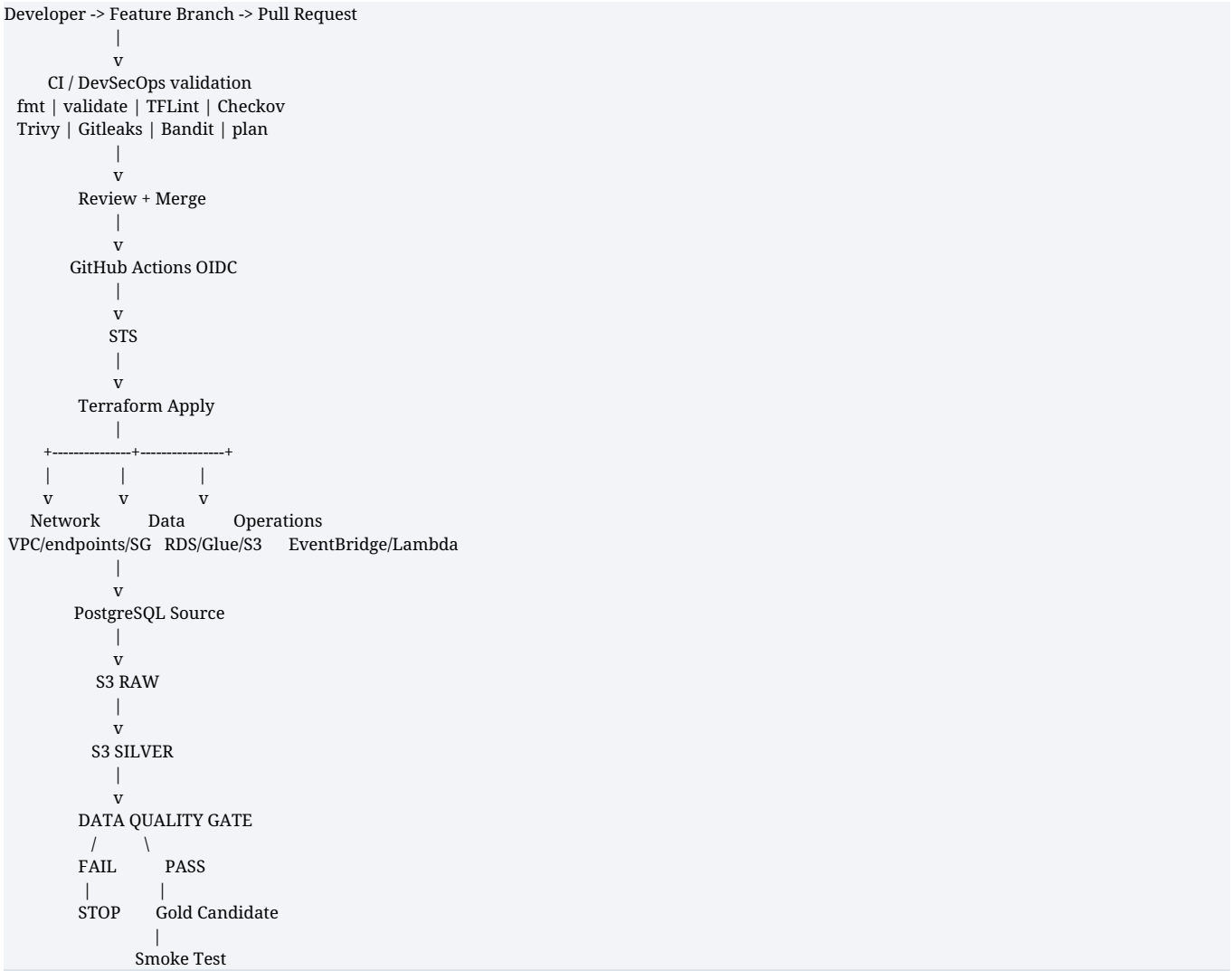
Fresh Build GitOps + CI/CD + IaC + DevSecOps + DataOps + AIOps

Complete 45-Step Hands-On Playbook v3

Rebuilt for a completely empty AWS lab - expanded Steps 36-42 live demonstrations

Purpose: Use this document as the single runbook from a blank AWS lab through GitHub OIDC, Terraform, private AWS infrastructure, Glue Raw/Silver processing, Athena DataOps gates, Git-versioned Gold releases, deliberately blocked bad releases, AIOps-assisted RCA, drift detection and cost-safe shutdown.

Target workflow



```
                    |
                    v
               warranty_public
                    |
               Tableau / AI / BI

Glue failure -> EventBridge -> AIOps Lambda -> Bedrock advisory RCA
Terraform drift -> scheduled plan -> detected change
```

What changed from v2

- Continuous numbering: every step from 1 through 45 is present.
- Fresh-build only: no dependency on yesterday's deleted VPC, RDS, S3, Glue jobs or crawlers.
- Explicit fixes for the two issues already encountered: wrong working directory for -chdir and literal YOUR_TF_STATE_BUCKET.
- Steps 36-37 now provide a full bad-release DataOps demonstration with exact Git, Python and Athena commands and proof that Gold promotion did not occur.
- Steps 38-41 now provide a complete AIOps demo including Bedrock precheck, a controlled JDBC failure, EventBridge/Lambda verification and safe restore.
- Step 42 now contains a reproducible Terraform drift demonstration rather than only explaining detailed-exitcode.
- Troubleshooting appendix maps common GitHub, Terraform, Glue, crawler, Athena and AIOps failures to exact next checks.

Section A - Prepare Git, GitHub and Terraform

Step 1 - Prepare the workstation

Use Git Bash, WSL, Linux, macOS or AWS CloudShell. You need Git, AWS CLI, Terraform and GitHub CLI.

- Authenticate AWS CLI with a sandbox administrator identity.
- Authenticate GitHub CLI with `gh auth login`.
- Use ap-south-1 consistently for the lab.

```
aws --version
terraform version
git --version
gh --version

aws sts get-caller-identity
gh auth status
```

Checkpoint: All four CLIs respond and AWS/GitHub authentication succeeds.

Step 2 - Create a new GitHub repository

- Create an empty repository named `aos-warranty-platform`.
- Do not initialize it with a conflicting README if you will push the supplied starter repository.

Checkpoint: The repository exists and you have push permission.

Step 3 - Extract the fresh-build starter ZIP

- Extract the fresh-build ZIP into a local directory.
- The repository root must directly contain `terraform/`, `.github/`, `glue/`, `scripts/`, `tests/`, `sql/`, `bootstrap/` and `aiops/`.

```
cd ~/aos-warranty-platform
pwd
ls -la
ls terraform
```

Expected: `ls terraform` shows files such as network.tf, rds.tf, glue.tf, catalog.tf, aiops.tf, variables.tf, outputs.tf and versions.tf.

Important: If you are already inside `terraform/`, run `terraform init` without `-chdir=terraform`. The error `chdir terraform: no such file or directory` means you ran the command from the wrong directory.

Step 4 - Push the starter to GitHub

```
git init
git add .
git commit -m "Initial fresh AOS warranty GitOps platform"
git branch -M main
git remote add origin https://github.com/YOUR_OWNER/aos-warranty-platform.git
git push -u origin main
```

Checkpoint: GitHub shows the complete repository structure at the repository root.

Step 5 - Create the GitHub dev environment

- GitHub -> Settings -> Environments -> New environment -> `dev`.
- If your GitHub plan supports it, configure a required reviewer to simulate a deployment approval gate.

Checkpoint: Environment `dev` exists.

Section B - One-time backend and GitHub OIDC bootstrap

Step 6 - Run the one-time bootstrap script

The S3 Terraform backend and GitHub OIDC role must exist before GitHub Actions can run Terraform.

```
export GITHUB_OWNER="YOUR_GITHUB_OWNER"
export GITHUB_REPO="aos-warranty-platform"
export AWS_REGION="ap-south-1"

bash bootstrap/bootstrap.sh
```

Expected: The script creates or reuses the S3 state bucket, configures the GitHub OIDC provider and `AOSGitHubDeployRole`, then creates repository variables.

Checkpoint: The script prints `Bootstrap completed`.

Step 7 - Verify the Terraform state bucket

```
aws s3 ls | grep aos-tfstate

gh variable list --repo "$GITHUB_OWNER/$GITHUB_REPO"
```

Expected: You see a real bucket such as `aos-tfstate-<account>-ap-south-1`; the variable `TF_STATE_BUCKET` contains that actual name.

Important: Never run Terraform with the literal placeholder `YOUR_TF_STATE_BUCKET`. S3 backend buckets must already exist before `terraform init`.

Step 8 - Initialize Terraform locally with the real backend

```
export TF_STATE_BUCKET=$(gh variable get TF_STATE_BUCKET   --repo "$GITHUB_OWNER/$GITHUB_REPO")

terraform -chdir=terraform init   -reconfigure   -backend-config="bucket=${TF_STATE_BUCKET}"
-backend-config="key=aos-warranty/dev/terraform.tfstate"   -backend-config="region=ap-south-1"   -backend-config="encrypt=true"   -backend-
config="use_lockfile=true"
```

Expected: Terraform reports that the S3 backend has been successfully configured.

Checkpoint: No `NoSuchBucket` error and no `YOUR_TF_STATE_BUCKET` text appears in the command.

Step 9 - Format and validate Terraform before PR

```
terraform -chdir=terraform fmt -recursive
terraform -chdir=terraform fmt -check -recursive -diff
terraform -chdir=terraform validate
```

Expected: `fmt -check` produces no diff and `validate` reports the configuration is valid.

Important: A PR that fails within a few seconds commonly fails at `terraform fmt -check`. Format locally, commit the changed .tf files and push again.

Step 10 - Run the GitHub OIDC-only test

- GitHub -> Actions -> `01 - OIDC Test` -> Run workflow.
- Inspect the `aws sts get-caller-identity` output.

Expected: The ARN contains `assumed-role/AOSGitHubDeployRole/...`.

Checkpoint: Do not troubleshoot Terraform, Glue or AWS deployment until OIDC succeeds.

Important: For repositories created after 15 July 2026, GitHub's default OIDC subject uses immutable owner and repository numeric IDs. The supplied bootstrap script builds the trust policy from those IDs.

Section C - PR GitOps and DevSecOps

Step 11 - Create the first feature branch

```
git checkout main
git pull
git checkout -b feature/initial-platform
```

Checkpoint: You are no longer working directly on main.

Step 12 - Make a harmless change and push the branch

```
echo "# GitOps validation" >> README.md
git add README.md
git commit -m "Validate GitOps control"
git push -u origin feature/initial-platform
```

Checkpoint: GitHub can create a pull request from the feature branch to main.

Step 13 - Open the Pull Request

- Open a PR to `main`.
- Do not merge until workflow `02 - PR CI DevSecOps` is green.

Checkpoint: The PR shows the required CI workflow.

Step 14 - Understand the PR gate sequence

- Terraform fmt checks canonical HCL formatting.
- OIDC obtains short-lived AWS credentials.
- Terraform init/validate checks provider/backend-aware syntax.
- TFLint checks Terraform quality.
- Checkov and Trivy scan IaC security.
- Bandit scans Python.
- Gitleaks checks secrets.
- Terraform plan previews infrastructure changes.

Checkpoint: You can identify which stage failed rather than treating every red PR as an AWS problem.

Step 15 - Fix the common fast PR formatting failure

```
terraform -chdir=terraform fmt -recursive
git status
git add terraform/
git commit -m "Format Terraform for CI"
git push
```

Expected: The same PR reruns automatically and the Terraform fmt step passes.

Step 16 - Demonstrate a DevSecOps block

Optionally introduce one temporary insecure Terraform change in a feature branch, such as a sensitive ingress rule open to 0.0.0.0/0.

- Push the intentionally insecure change.
- Observe Checkov or Trivy fail the PR.
- Remove the insecure change and push again.

Expected: The PR is red while insecure IaC is present and green after the correction.

Important: Do not disable the entire scanner to make a lab pass. If a disposable-lab exception is genuinely required, document and scope the exception to the specific check.

Step 17 - Merge only after PR checks pass

- Review the Terraform plan.
- Approve the PR.
- Merge to `main`.

Checkpoint: Git history records who proposed, reviewed and merged the infrastructure change.

Section D - Fresh AWS infrastructure build

Step 18 - Run the one-time Fresh Lab Bootstrap workflow

- GitHub -> Actions -> `03 - Fresh Lab Bootstrap` -> Run workflow.
- Approve the `dev` environment if GitHub pauses for approval.

Expected: Terraform creates the complete lab and the workflow proceeds to seed PostgreSQL and run the data pipeline.

Step 19 - Verify the Terraform-created VPC

- VPC name: `aos-lab-vpc`.
- CIDR: `10.20.0.0/16`.
- Two private subnets: `10.20.10.0/24` and `10.20.20.0/24`.
- DNS support and DNS hostnames enabled.

Checkpoint: No public subnet/NAT dependency is required for the lab data path.

Step 20 - Verify security groups

- `aos-glue-sg`: self-referencing TCP for Glue/Spark workers.
- `aos-rds-sg`: PostgreSQL 5432 from `aos-glue-sg` only.
- `aos-vpce-sg`: HTTPS 443 from `aos-glue-sg`.

Checkpoint: RDS is not open to 0.0.0.0/0.

Step 21 - Verify private AWS service connectivity

- STS interface endpoint with Private DNS.
- Secrets Manager interface endpoint with Private DNS.
- Glue interface endpoint with Private DNS.
- S3 gateway endpoint associated with the private route table.

Checkpoint: All endpoints are Available.

Step 22 - Verify the S3 data-lake bucket

- Bucket name is generated from the AWS account and Region.
- Block Public Access is enabled.
- Server-side encryption is enabled.

Checkpoint: The bucket exists and is private.

Step 23 - Verify private RDS PostgreSQL

- DB identifier: `aos-sap-simulator`.
- Database: `aosdb`.
- Public access: false.
- Security group: `aos-rds-sg`.
- RDS manages the master password in Secrets Manager.

Checkpoint: RDS state is Available before Glue seed/ingestion begins.

Step 24 - Verify the Glue runtime role

- Role: `AWSGlueServiceRole-AOSFreshLab`.
- AWSGlueServiceRole managed policy attached.
- S3 data-lake access.
- GetSecretValue/DescribeSecret on the RDS-managed secret.

Checkpoint: Glue can read its scripts/data and obtain DB credentials without storing a password in Git.

Step 25 - Verify the Glue NETWORK connection

The fresh build uses a Glue NETWORK connection only to attach Glue workers to the correct private subnet/security group.

- Connection name: `aos-postgres-network`.
- Subnet: private subnet A.
- Security group: `aos-glue-sg`.

Checkpoint: Glue jobs are VPC-attached and can reach RDS and private AWS service endpoints.

Step 26 - Verify all three Glue jobs

- `aos-seed-postgres` - seeds source tables.
- `aos-ingest-raw` - reads four PostgreSQL tables and writes S3 Raw.
- `aos-transform-warranty-silver` - joins/enriches Raw data and calculates warranty status.

Checkpoint: All jobs use Glue 5.0 and the Terraform-created Glue role.

Step 27 - Verify Data Catalog databases and crawlers

- `aos_warranty_raw` + `aos-raw-crawler`.
- `aos_warranty_silver` + `aos-silver-crawler`.
- `aos_warranty_gold` for versioned Gold views.

Checkpoint: The Raw crawler targets `s3://<data-bucket>/raw/`; the Silver crawler targets `s3://<data-bucket>/silver/warranty/`.

Section E - First end-to-end data pipeline

Step 28 - Upload version-controlled Glue scripts

The bootstrap workflow copies the repository's Glue scripts into the Terraform-created data bucket.

```
BUCKET=$(terraform -chdir=terraform output -raw data_bucket_name)

aws s3 cp glue/seed_source.py   "s3://${BUCKET}/scripts/seed_source.py"

aws s3 cp glue/ingest_raw.py   "s3://${BUCKET}/scripts/ingest_raw.py"

aws s3 cp glue/transform_silver.py   "s3://${BUCKET}/scripts/transform_silver.py"
```

Checkpoint: S3 `scripts/` contains the three current Git-controlled scripts.

Step 29 - Seed the PostgreSQL source automatically

`aos-seed-postgres` creates/overwrites the four demo source tables using Spark JDBC and the RDS-managed secret.

- customers
- materials
- sales_orders
- warranty_claims

Expected: The job logs `SOURCE SEED COMPLETE` and the four table row counts.

Step 30 - Run Raw ingestion

`aos-ingest-raw` reads PostgreSQL and writes Parquet to S3.

Expected: S3 contains `raw/customers/`, `raw/materials/`, `raw/sales_orders/` and `raw/warranty_claims/` with Parquet files.

Step 31 - Run and validate the Raw crawler

```
SHOW TABLES IN aos_warranty_raw;

SELECT * FROM aos_warranty_raw.customers LIMIT 10;
SELECT * FROM aos_warranty_raw.materials LIMIT 10;
SELECT * FROM aos_warranty_raw.sales_orders LIMIT 10;
SELECT * FROM aos_warranty_raw.warranty_claims LIMIT 10;
```

Checkpoint: Athena returns all four Raw tables.

Important: If the crawler succeeds but adds zero tables, first verify that Parquet objects actually exist under the exact S3 crawler path.

Step 32 - Run the Silver transformation

The Silver job joins claims -> sales orders -> customers -> materials and calculates the exact warranty expiry date.

Expected: WC001 is out of warranty because sale date 2018-01-01 + 6 years = 2024-01-01, while claim date is 2025-02-10.

Step 33 - Run and validate the Silver crawler

```
SHOW TABLES IN aos_warranty_silver;

SELECT
 claim_id,
 customer_name,
 material_description,
 sale_date,
 claim_date,
```

```
  warranty_expiry_date,
  warranty_status
FROM aos_warranty_silver.warranty
ORDER BY claim_id;
```

Checkpoint: Silver table exists and the warranty calculation is visible in Athena.

Step 34 - Run the automated DataOps quality gates

- `orphan_claims.sql` -> violation_count must equal 0.
- `negative_claims.sql` -> 0.
- `claim_before_sale.sql` -> 0.
- `missing_material.sql` -> 0.

Expected: All four tests return `violation_count=0`; otherwise the workflow exits non-zero before Gold creation.

Step 35 - Create, smoke test and promote the first Gold release

The pipeline derives a version from the Git commit SHA, creates a versioned Gold view, smoke tests it and only then updates `warranty_public`.

```
SHOW CREATE VIEW aos_warranty_gold.warranty_public;

SELECT *
FROM aos_warranty_gold.warranty_public
ORDER BY claim_id;
```

Checkpoint: Record the version currently backing `warranty_public`. This is your last-known-good release for Step 36.

Section F - Expanded Steps 36-42: Live demonstrations

Why these steps matter: Steps 36-42 are designed as deliberate controlled demonstrations. A red workflow in Step 37 or a failed Glue job in Step 39 is a successful test when it proves that the platform blocks bad data or detects operational faults without corrupting the last-known-good consumer release.

Step 36 - Demonstrate DataOps blocking a bad release - prepare and inject bad data

Create a controlled Git change that injects one orphan warranty claim into the Raw ingestion output. The source RDS remains valid; only this candidate release carries the bad record.

- First record the current `warranty_public` backing version using Athena.
- Create branch `demo/dataops-bad-release`.
- Edit `glue/ingest_raw.py` only in that branch.
- Inject claim `WC999` with order_id `SO_DOES_NOT_EXIST` only when processing `warranty_claims`.

```
-- ATHENA: record last-known-good version
SHOW CREATE VIEW aos_warranty_gold.warranty_public;

-- GIT
git checkout main
git pull
git checkout -b demo/dataops-bad-release
```

Checkpoint: You know the current Gold version before the bad change.

Step 36A - Exact code to add to glue/ingest_raw.py

Add these imports near the top:

```
from datetime import date
from decimal import Decimal
```

Immediately after the existing `spark.read.jdbc(...)` block inside the table loop, add:

```
# DEMO ONLY - deliberately inject one orphan claim.
if table == "warranty_claims":

  bad_claim = spark.createDataFrame(
    [
      (
        "WC999",
        "SO_DOES_NOT_EXIST",
        date(2025, 1, 1),
        Decimal("500.00"),
        "DEMO_BAD_DATA"
      )
    ],
    schema=df.schema
  )

  df = df.unionByName(bad_claim)

  print("DEMO: Injected orphan warranty claim WC999")
```

Then commit and push:

```
git add glue/ingest_raw.py
git commit -m "Demo DataOps gate with orphan warranty claim"
git push -u origin demo/dataops-bad-release
```

Expected: The PR's code/IaC security checks can still pass because this is syntactically valid code. That is intentional: DevSecOps validates code/security, while DataOps validates runtime data correctness.

Step 37 - Execute the bad release, prove the block, then recover

Merge the controlled demo change, let the normal deployment reach the DataOps gate, and prove that `warranty_public` never moved to the bad release.

- Open PR `demo/dataops-bad-release` -> `main`.
- Let `02 - PR CI DevSecOps` pass.
- Merge the PR.
- Watch `04 - Deploy Data Platform` execute Raw -> crawler -> Silver -> crawler -> DataOps.
- The expected failure is `tests/orphan_claims.sql => violation_count=1` followed by `DATA QUALITY GATE FAILED` and exit code 2.
- Do not treat the red deployment as a problem; it is the control working.

Expected: The pipeline stops before `Create versioned Gold view`, `Smoke test candidate` and `Promote warranty_public`.

Checkpoint: A bad candidate reached Raw/Silver, but the consumer-facing Gold release stayed unchanged.

Step 37A - Prove the bad record exists in Raw

```
SELECT *
FROM aos_warranty_raw.warranty_claims
WHERE claim_id = 'WC999';
```

Expected row: `WC999 | SO_DOES_NOT_EXIST | ... | DEMO_BAD_DATA`.

Step 37B - Prove the DataOps rule caught it

```
SELECT
  w.claim_id,
  w.order_id
FROM aos_warranty_raw.warranty_claims w
LEFT JOIN aos_warranty_raw.sales_orders s
  ON w.order_id = s.order_id
WHERE s.order_id IS NULL;
```

Expected: exactly the orphan record `WC999 / SO_DOES_NOT_EXIST`.

Step 37C - Prove Gold was NOT promoted

```
SHOW CREATE VIEW aos_warranty_gold.warranty_public;
```

Compare the view definition with the last-known-good version recorded in Step 35. It must still point to the previous good Git-SHA view.

Presentation line: CI and DevSecOps accepted the code, but the runtime DataOps gate detected invalid business data. The release stopped before Gold promotion, so Tableau/AI consumers remained on the last-known-good `warranty_public` contract.

Step 37D - Fix and prove recovery

Remove the demo injection using either a revert or a fix branch. A clean demonstration uses a new fix PR:

```
git checkout main
git pull
git checkout -b fix/remove-dataops-demo

# Edit glue/ingest_raw.py and remove:
# - datetime/date imports added only for the demo
# - Decimal import added only for the demo
# - the WC999 injection block

git add glue/ingest_raw.py
git commit -m "Remove DataOps bad-record simulation"
git push -u origin fix/remove-dataops-demo
```

Open the fix PR, let CI pass, merge, and watch `04 - Deploy Data Platform` run again.

```
SELECT *
FROM aos_warranty_raw.warranty_claims
WHERE claim_id = 'WC999';

SHOW CREATE VIEW aos_warranty_gold.warranty_public;
```

Expected recovery: Because Raw writes use overwrite, WC999 disappears; all four DQ counts return 0; a new Git-SHA Gold

view is created and `warranty_public` is promoted to the new successful version.

Step 38 - Verify Bedrock before the AIOps failure demo

Do not deliberately fail Glue until you know the AIOps Lambda has a usable Bedrock model/inference profile.

- Open Amazon Bedrock in ap-south-1 and verify model access/inference availability for the configured model/profile.
- The starter uses the Terraform variable `bedrock_model_id`; change it if your account/Region requires another supported profile.
- The Lambda role must allow `bedrock:InvokeModel`.

```
terraform -chdir=terraform output
aws lambda get-function-configuration   --function-name aos-aiops-rca   --query 'Environment.Variables'
```

Checkpoint: `BEDROCK_MODEL_ID` is populated and you have a supported inference target.

Important: Bedrock model/profile availability can vary by account and Region. If the model invocation fails, the Lambda still logs the Glue error plus the Bedrock invocation error; fix model access before evaluating AIOps quality.

Step 39 - Create a safe, known Glue failure for AIOps

Temporarily break exactly one known network dependency: PostgreSQL 5432 from Glue to RDS.

- AWS Console -> EC2 -> Security Groups -> `aos-rds-sg`.
- Record the inbound PostgreSQL rule before changing it.
- Temporarily remove the rule `TCP 5432` with source `aos-glue-sg`.
- Run Glue job `aos-ingest-raw` manually.

Expected: The Glue job eventually enters FAILED with a JDBC/network connection error.

Checkpoint: You can see the failed JobRun ID and its ErrorMessage.

Important: Do not delete the RDS instance, subnet, VPC endpoints or Terraform state. This test changes only one SG rule and must be restored immediately afterwards.

Step 39A - CLI alternative for starting the failure

```
RUN_ID=$(aws glue start-job-run   --job-name aos-ingest-raw   --query JobRunId   --output text)

echo "$RUN_ID"

aws glue get-job-run   --job-name aos-ingest-raw   --run-id "$RUN_ID"   --query 'JobRun.{State:JobRunState,Error:ErrorMessage}'   --output json
```

Step 40 - Observe EventBridge -> Lambda -> Bedrock AIOps RCA

The failure should generate a Glue Job State Change event. EventBridge routes matching FAILED/TIMEOUT/STOPPED events to `aos-aiops-rca`.

- Open CloudWatch -> Log groups -> `/aws/lambda/aos-aiops-rca`.
- Open the newest log stream.
- Confirm the Lambda retrieved the Glue JobRun error.
- Confirm Bedrock generated or attempted to generate an advisory RCA.

Expected: The log contains probable root cause, confidence, affected layer, first checks, safe remediation and rollback guidance. For the deliberate test, it should point toward RDS/JDBC/security-group connectivity.

Checkpoint: AIOps transformed a raw service failure into an operator-focused diagnostic recommendation.

Step 40A - Verify the EventBridge rule if Lambda did not run

```
aws events describe-rule   --name aos-glue-failure

aws events list-targets-by-rule   --rule aos-glue-failure

aws lambda get-policy   --function-name aos-aiops-rca
```

Interpretation: Glue sends `Glue Job State Change` service events directly to EventBridge on a best-effort basis. The rule must match source `aws.glue`, detail-type `Glue Job State Change`, the lab job names, and failed states.

Step 41 - Restore service safely and keep AIOps human-gated

AIOps recommends; a human validates and restores the known-good infrastructure control.

- Restore RDS inbound TCP/5432 with source `aos-glue-sg`.
- Run `aos-ingest-raw` again.
- Confirm the job succeeds.
- Run `terraform plan` afterwards: because you manually restored the Terraform-declared rule, there should be no remaining drift.

Expected: Glue succeeds again. No AI component directly modifies the SG.

Checkpoint: You can demonstrate the operating model `detect -> analyze -> recommend -> human approve -> remediate -> verify`.

Important: Do not give the AIOps Lambda broad EC2/IAM permissions merely to make the demo autonomous. Advisory RCA is the safer baseline.

Step 42 - Demonstrate Terraform drift detection with a reversible manual change

Create one harmless manual drift, detect it with workflow `05 - Terraform Drift`, then return AWS to Git desired state.

- Choose a low-risk Terraform-managed tag rather than a network rule for this demo.
- For example, add a manual tag `DriftDemo=true` to the Terraform-managed S3 data bucket in the AWS Console.
- GitHub -> Actions -> `05 - Terraform Drift` -> Run workflow.
- Terraform plan uses `-detailed-exitcode`; exit 2 is interpreted by the workflow as drift and turns the run red.

Expected: The workflow logs `DRIFT DETECTED` and shows the out-of-band tag difference.

Checkpoint: The red drift workflow proves AWS no longer matches the Git/Terraform desired state.

Step 42A - Reconcile the drift

Use one of these two GitOps-correct choices:

- If the manual change was unauthorized/unwanted: remove the manual tag in AWS, then rerun drift and expect exit 0.
- If the change is genuinely desired: add the tag to Terraform, open a PR, pass DevSecOps/plan review, merge and apply. Do not silently accept console drift.

```
# After restoring/reconciling:
terraform -chdir=terraform plan -detailed-exitcode

# Meaning:
# 0 = no changes / no drift
# 1 = Terraform error
# 2 = changes/drift detected
```

Presentation line: Git is the desired state. Manual console changes are detectable; legitimate changes must return through PR review and Terraform so the control plane and runtime stay aligned.

Section G - Cost control, cleanup and repeatability

Step 43 - Stop RDS when you are not demonstrating

- GitHub -> Actions -> `06 - RDS Cost Control`.
- Choose STOP after your session and START before the next live ingestion demo.

Expected: RDS configuration and data remain; compute stops while the DB is stopped. Storage charges still apply.

Important: Interface VPC endpoints continue to incur endpoint-hour charges while provisioned. For a short-lived demo, keeping them avoids rebuild risk; for longer idle periods, destroy the whole disposable lab.

Step 44 - Destroy the disposable lab when finished

- GitHub -> Actions -> `07 - Destroy Lab`.
- Enter confirmation `DESTROY`.
- Terraform destroys the resources it manages.

Checkpoint: The remote Terraform state bucket intentionally remains so state history/backend configuration is not lost.

Important: The lab configuration uses `force_destroy` for the data bucket and `skip_final_snapshot` for RDS. This is appropriate only for the disposable simulation, not production.

Step 45 - Rebuild later from Git rather than manually

- Ensure `LAB_BOOTSTRAPPED=false` before a completely fresh bootstrap if needed.
- Run `03 - Fresh Lab Bootstrap` again.
- Terraform recreates the environment from Git.
- The seed job recreates source data and the pipeline rebuilds Raw/Silver/Gold.

Checkpoint: You can reproduce the entire architecture without manually reconstructing VPC, RDS, IAM, Glue or data layers.

Troubleshooting decision matrix

Symptom	Likely cause	Exact next action
`chdir terraform: no such file or directory`	Wrong current directory.	Run `pwd`/`ls`. From repo root use `terraform -chdir=terraform ...`; from inside terraform/ use `terraform ...`.
S3 backend says `YOUR_TF_STATE_BUCKET` does not exist	Placeholder was used literally.	Run bootstrap.sh or create the real bucket, then init with `-reconfigure` and the actual bucket variable.
PR fails in ~5-10 seconds	Usually Terraform fmt check.	`terraform -chdir=terraform fmt -recursive`; commit formatted .tf files; push.
`AssumeRoleWithWebIdentity` denied	GitHub OIDC trust subject/audience/role mismatch.	Run `01 - OIDC Test`; confirm immutable owner/repo IDs for new repositories and `aud=sts.amazonaws.com`.
Checkov/Trivy fails	IaC security finding.	Fix the control or add only a scoped documented lab exception. Do not globally disable the scanners.
Glue cannot assume role / STS access	Private STS path or SG/Private DNS issue.	Check STS interface endpoint = Available, Private DNS enabled, VPCE SG allows 443 from Glue.
Glue JDBC connection fails	RDS endpoint/secret/5432/SG/VPC issue.	Check RDS Available, Glue SG self-reference, RDS SG 5432 from Glue SG, secret access, job NETWORK connection.
Crawler succeeds but zero tables	No Parquet under path or crawler S3/config issue.	List exact S3 prefix recursively; verify role read permissions and exact crawler source path.
Athena TABLE_NOT_FOUND	Glue Catalog table missing/wrong name.	`SHOW TABLES IN <db>` and use the actual crawler-created name.
Bad release does not fail DQ	Injection did not reach Raw or test query points at wrong DB/table.	Query WC999 directly in Raw, then run orphan_claims.sql manually.
Bad release fails but Gold changed	Promotion occurred before DQ in workflow.	Move all Gold create/promote steps after the DQ step; GitHub stops subsequent steps after non-zero exit.
AIOps Lambda does not run	EventBridge pattern/target/Lambda permission mismatch.	Describe rule, list targets, check Lambda resource policy and Glue job name/state filters.
AIOps Lambda runs but Bedrock fails	Model/profile access or Region issue.	Inspect Lambda log; set a supported Bedrock inference profile/model and verify `bedrock:InvokeModel`.
Drift workflow returns 2	Terraform sees a change.	Expected for the demo; inspect plan. Revert manual AWS drift or capture the desired change through Git/PR.

Live demonstration checklist

1. OIDC Test green - prove no long-lived AWS keys.
2. PR CI green - show Terraform plan and DevSecOps gates.
3. Fresh Lab Bootstrap green - show Terraform-created VPC/RDS/S3/Glue.
4. Athena Raw and Silver queries working.
5. Record current warranty_public last-known-good version.
6. Bad-data PR passes code security checks.
7. Deployment goes red at DataOps with orphan_claims=1.
8. Athena proves WC999 exists in Raw.
9. SHOW CREATE VIEW proves warranty_public did not move.
10. Fix PR removes WC999; pipeline green; public view promoted.
11. Remove RDS 5432 rule and run Glue to create a controlled failure.
12. CloudWatch shows EventBridge/Lambda/Bedrock RCA.
13. Restore 5432 rule and prove Glue succeeds.
14. Create harmless manual tag drift; run 05 - Terraform Drift; observe exit 2.
15. Reconcile drift and rerun to green.

16. Stop RDS or destroy the lab after the demo.

Senior-architect explanation

60-second narrative: Git is the source of truth for infrastructure, ETL, quality rules and business-product releases. Pull requests enforce IaC and code security before deployment. GitHub Actions uses OIDC and short-lived STS credentials, Terraform creates the private AWS platform, Glue builds Raw and Silver datasets, and Athena quality tests act as release gates. A bad-data candidate can reach Raw/Silver but cannot promote Gold, so consumers remain on the last-known-good public view. Runtime Glue failures are routed through EventBridge to an AIOps Lambda that uses Bedrock for advisory RCA, while Terraform drift detection identifies out-of-band console changes. Human approval remains in the remediation loop.

Current reference notes

- GitHub OIDC: repositories created after 15 July 2026 use immutable default subject claims containing owner and repository IDs.
- Terraform S3 backend: native state locking is enabled with `use_lockfile=true`; DynamoDB-based locking is deprecated.
- AWS Glue emits `Glue Job State Change` events directly to Amazon EventBridge on a best-effort basis.
- Amazon Bedrock supports inference profiles from ap-south-1; exact model/profile availability must be verified for the account before the AIOps demo.